

第 4 讲

Compute Architecture and Scheduling

目录

1 核心内容	1
1.1 软件执行层次	1
1.2 线程块的三个基本语义	1
1.3 软件层次到硬件层次的映射	2
2 GPU 计算架构与线程块驻留	2
2.1 流式多处理器是主要硬件组织单元	2
2.2 线程块以整体形式分配到一个流式多处理器	3
2.3 为什么同一线程块必须留在同一个流式多处理器	4
2.4 驻留需要同时满足多类资源限制	5
3 线程块内同步、跨线程块同步与可伸缩性	5
3.1 线程块级屏障的语义	5
3.2 发散条件中的屏障为什么危险	6
3.3 为什么线程块必须一次性获得完整驻留资源	6
3.4 普通线程块之间为何不能依赖执行顺序	7
3.5 跨线程块屏障的死锁风险	7
4 线程束、处理块与单指令多线程执行	8
4.1 线程块进一步划分为线程束	8
4.2 处理块与线程束调度器	9
4.3 从单程序多数据到单指令多线程	10
4.4 独立线程调度不改变性能分析的基本单位	10
4.5 驻留线程束、就绪线程束与实际发射线程束	11
5 控制发散与有效执行通道	11
5.1 控制发散是线程束内部的问题	11
5.2 分支发散示例	11
5.3 活动通道比例与发散区间效率	12
5.4 循环也会产生控制发散	13
5.5 通过线程束粒度组织条件减少发散	13
5.6 边界检查中的少量发散通常不可避免	14
6 线程束调度与延迟隐藏	14
6.1 线程束调度器选择可执行的线程束	14
6.2 长延迟操作如何造成线程束停顿	15
6.3 用其他线程束覆盖等待时间	15
6.4 为什么驻留线程数远大于物理核心数	16
6.5 延迟隐藏失效的典型情形	16
7 占用率、资源约束与线程块大小	17
7.1 占用率的定义	17
7.2 流式多处理器资源示例	17
7.3 把多重驻留限制写成统一形式	18
7.4 H100 风格限制下的三个线程块大小	19

7.5	二维线程块大小：彩色转灰度核函数	20
7.6	占用率高并不保证性能最高	21
8	运行时资源查询与例题	21
8.1	运行时查询设备属性	21
8.2	占用率计算工具	22
8.3	例题一：1536 线程与 4 线程块上限	22
8.4	例题二：只知道流式多处理器和核心数，能否选出最佳配置	23
8.5	例题三：统一分支条件是否造成控制发散	24
9	架构、调度与性能机制的统一关系	24
9.1	从网格到执行通道的完整映射	24
9.2	四个性能概念之间的因果链	25
9.3	概念边界总结	25

1 核心内容

内容概览

- 理解 GPU 架构与组织的基础；
- 理解线程调度、延迟容忍与硬件占用率；
- 区分驻留线程束与当前实际发射指令的线程束。

1.1 软件执行层次

CUDA 的软件执行层次由网格、线程块和线程组成。网格中的所有线程运行同一份核函数代码，每个线程通过线程索引和线程块索引选择自身负责的数据与工作。这种执行方式属于单程序多数据 (Single Program, Multiple Data, SPMD) 模型。

线程块可以组织成一维、二维或三维形状。多维组织使线程索引能够自然对应多维数据，从而简化地址计算；维度本身并不会自动提高访存效率或计算性能。例如，二维图像既可以使用二维线程块，也可以使用一维线程块，后一种方式需要显式完成一维索引到二维坐标的映射。

关键结论

线程块的形状用于表达问题结构；硬件执行方式取决于线程块如何驻留到流式多处理器、如何拆分为线程束，以及线程束如何被调度。

1.2 线程块的三个基本语义

1. 线程块是资源分配与驻留的基本单位。同一线程块的全部线程被分配到同一个流式多处理器；一个线程块不会跨越多个流式多处理器执行。
2. 同一线程块内部可以高效协作。线程可以使用共享内存，并通过线程块级屏障进行同步。
3. 不同线程块的执行顺序没有保证。核函数必须在不依赖线程块调度顺序的前提下得到正确结果；同一程序由此能够在硬件并行度不同的 GPU 上透明伸缩。

1.3 软件层次到硬件层次的映射

软件执行层次与硬件执行资源之间的基本关系为

网格 → 线程块 → 流式多处理器驻留 → 线程束 → 处理块发射与执行。

线程束(warp)是硬件调度的基本单位。线程块驻留到某个流式多处理器后，其线程按照线性化线程索引划分为若干线程束；截至 Hopper 的所列 NVIDIA 架构均采用 32 线程的线程束。控制发散、延迟隐藏与占用率都以线程束为核心进行分析。

2 GPU 计算架构与线程块驻留

2.1 流式多处理器是主要硬件组织单元

一块 GPU 由多个**流式多处理器(Streaming Multiprocessor, SM)**组成。每个流式多处理器内部包含若干执行核心、控制资源和片上存储资源；多个流式多处理器通过更高层次的存储系统访问全局内存(global memory)。以 Hopper H100 的架构示意图为例，144 个流式多处理器、每个流式多处理器 128 个核心，对应

$$144 \times 128 = 18432$$

个核心。

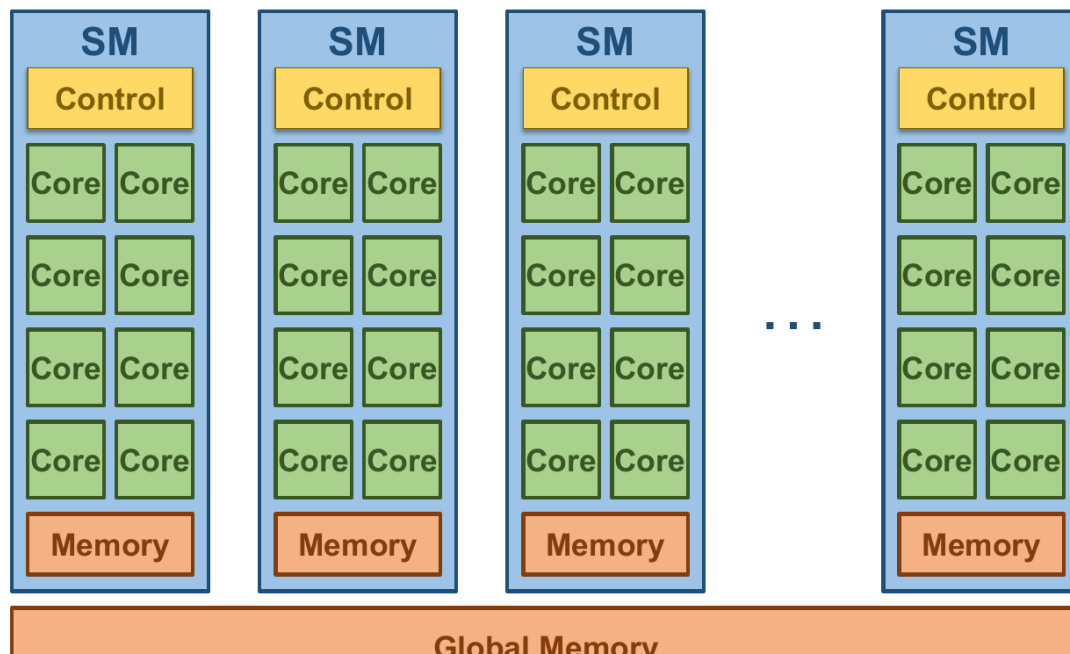


图 1 多个流式多处理器共享更高层次的全局内存；每个流式多处理器内部具有控制、执行核心和片上存储资源

定义

流式处理器 (Streaming Processor, SP) 或 CUDA 核心表示流式多处理器内部的标量执行核心。流式多处理器负责维护大量驻留线程的执行状态并调度线程束；流式处理器是实际完成相应算术工作的执行资源之一。两者不能用同一个“核心数”概念替代。

2.2 线程块以整体形式分配到一个流式多处理器

CUDA 线程不是逐个、独立地被分配到任意流式多处理器。硬件以**线程块粒度**完成驻留分配：一个线程块的全部线程都属于同一个流式多处理器，并在该流式多处理器上保持驻留直到线程块完成。一个线程块不能拆到多个流式多处理器上。

多个线程块可以同时驻留在同一个流式多处理器上，但前提是它们共同占用的硬件资源没有超过该流式多处理器的限制。尚未获得驻留资源的线程块处于等待状态；当已有线程块完成并释放资源后，等待线程块才可能获得驻留机会。

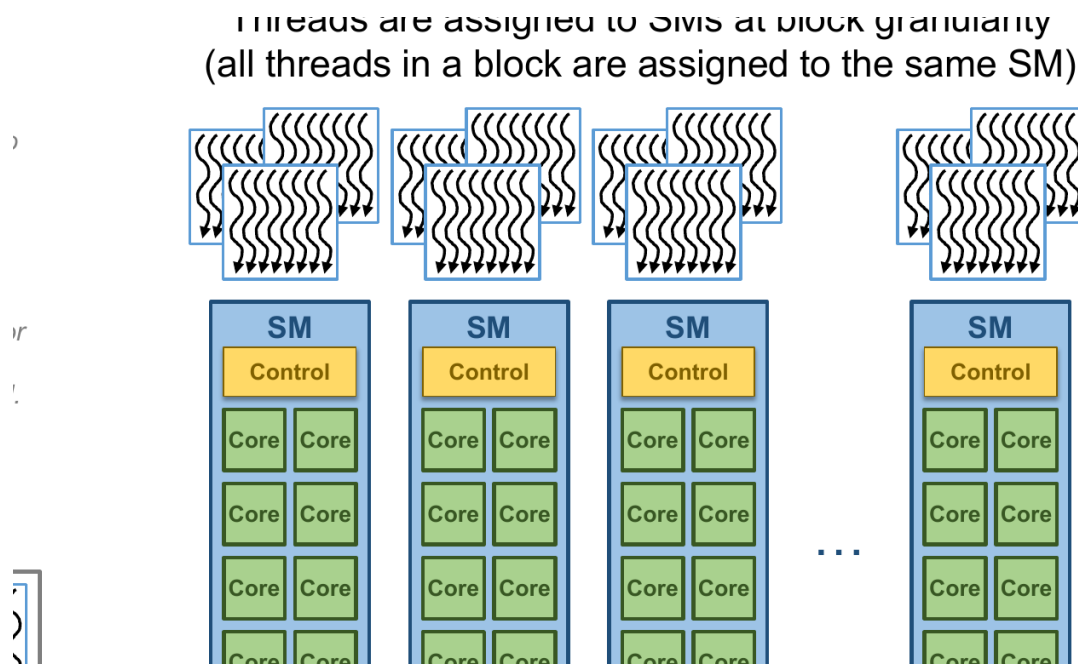


图 2 线程块以整体形式分配到流式多处理器；超出当前驻留能力的线程块必须等待资源释放

这里必须区分**驻留**和**正在执行**。假设某个流式多处理器能够容纳 2048 个驻留线程，并不意味着这 2048 个线程在同一个时钟周期中都各自占用一个物理执行核心。一个流式多处理器的物理执行核心数通常远小于可驻留线程数；驻留线程会进一步组成线程束，由调度器选择其中已经具备执行条件的线程束发射指令。

关键结论

“线程块被分配到某个流式多处理器”表示该线程块所需的执行状态和资源已在该流式多处理器上建立，并不表示线程块中的所有线程同时占用独立核心执行。

2.3 为什么同一线程块必须留在同一个流式多处理器

同一线程块中的线程依赖两类低开销协作资源：

- **共享内存 (shared memory)**: 物理资源位于流式多处理器上，但逻辑上按线程块分配；同一线程块中的线程可以访问该线程块的共享内存。
- **线程块级屏障**: 同一线程块的线程可以使用 `__syncthreads()` 等机制等待彼此达到同步点。

如果一个线程块被拆分到不同流式多处理器，这些低开销机制就失去统一的本地硬件基础。因此，线程块边界既是编程模型中的协作边界，也是硬件资源分配的重要边界。

不同线程块之间如果需要交换数据，通常必须依赖全局可见的存储与更高层次的同步机制。其成本和约束都显著高于线程块内部协作，因此 CUDA 的基本执行模型要求普通线程块之间不依赖具体的调度顺序。

2.4 驻留需要同时满足多类资源限制

线程块获得驻留位置之前，流式多处理器必须能够为整个线程块预留足够资源。主要约束包括：

- 每线程块最大线程数；
- 每流式多处理器最大驻留线程数；
- 每流式多处理器最大驻留线程块数；
- 寄存器文件容量；
- 共享内存容量；
- 最大驻留线程束数等架构限制。

因此，“线程数还放得下”只是必要条件之一。例如，一个线程块可能在线程数方面合法，但其寄存器或共享内存需求使得同一个流式多处理器只能驻留很少的线程块。后面的占用率分析将把这些约束统一起来。

3 线程块内同步、跨线程块同步与可伸缩性

3.1 线程块级屏障的语义

同一线程块内最基本的屏障同步由 `__syncthreads()` 提供。其核心语义是：到达该屏障的线程必须等待，直到线程块中所有尚未退出且应参与该屏障的线程都一致地到达同一个屏障实例，然后才能继续执行。

机制

线程块级屏障之所以可以高效实现，是因为同一线程块的全部驻留状态位于同一个流式多处理器中。屏障除了协调控制进度，还会对相关的共享内存和全局内存访问建立必要的顺序关系，使屏障之前的相关写入在屏障之后按照该同步语义被观察。

3.2 发散条件中的屏障为什么危险

下面的结构存在根本问题：

代码 1 发散条件中的两个不同屏障实例

```
1 if (predicate) {  
2     __syncthreads();  
3 } else {  
4     __syncthreads();  
5 }
```

表面上看，两条分支都出现了 `__syncthreads()`；但如果线程块中的不同线程沿不同分支前进，它们到达的可能是两个不同的静态屏障实例。这样无法保证全部参与线程在同一个屏障上会合，结果可能造成错误甚至死锁。

正确性要求可以概括为：对于某一个线程块级屏障实例，所有应参与的线程必须以兼容、可达且一致的方式到达它。屏障不能依赖会让同一线程块中的线程分成互不相交路径的控制结构。

需要特别注意

“每条分支里都写一个屏障”不等价于“所有线程到达同一个屏障”。判断同步是否正确，必须看线程是否会合到同一个屏障实例，而不是只看源代码中是否都出现了同名函数。

3.3 为什么线程块必须一次性获得完整驻留资源

线程块不能只让一部分线程先驻留，剩余线程等以后再分配。原因可以用屏障直接解释：假设线程块前半部分已经驻留并执行到屏障，而后半部分因为资源不足根本无法开始执行，那么前半部分会永久等待后半部分；同时前半部分占据的资源又可能阻止后半部分获得驻留机会，从而形成死锁。

所以，线程块只有在流式多处理器能够为**整个线程块**提供所需驻留资源时，才会被接纳。资源分配的整体性是线程块级同步语义能够成立的硬件条件之一。

3.4 普通线程块之间为何不能依赖执行顺序

如果不同线程块之间没有同步依赖，那么这些线程块可以：

- 以任意顺序开始；
- 在不同流式多处理器上并行执行；
- 在硬件资源较少时部分顺序执行；
- 在硬件资源较多时让更多线程块同时执行。

这形成了**透明可伸缩性** (*transparent scalability*)：同一份核函数代码可以在具有不同流式多处理器数量的 GPU 上运行，而正确性不依赖具体有多少线程块同时驻留。

关键结论

线程块执行顺序不受程序员控制，因此普通核函数中的正确性条件必须对所有可能的线程块调度顺序成立。

3.5 跨线程块屏障的死锁风险

若程序试图让整个网格中的线程块通过普通自制屏障互相等待，问题在于部分线程块可能尚未获得驻留。已经驻留的线程块先到达屏障并占用流式多处理器资源；未驻留的线程块却必须先运行到同一屏障才能解除等待。如果驻留资源被等待中的线程块占满，后续线程块就永远没有机会启动。

受约束的跨线程块同步机制主要包括：

- **协作组** (*cooperative groups*) 可以在满足特定驻留约束的条件下，支持线程块簇或整个网格范围的屏障；关键前提是相关参与者能够被同时调度到硬件上。
- 某些单向生产者—消费者同步可以通过更严格的执行结构建立先后关系，但普通线程块的调度顺序本身仍不能被程序假定。

因此，跨线程块同步属于比线程块内同步更高成本、约束更多的机制。除非算法明确需要并满足相应条件，否则应优先保持线程块之间的顺序独立性。

4 线程束、处理块与单指令多线程执行

4.1 线程块进一步划分为线程束

线程块驻留到流式多处理器后，硬件按照线程的线性化索引把它拆分成若干线程束(warp)。线程束大小由设备实现决定；截至 Hopper 的所列 NVIDIA 架构均采用 32 个线程。

若令线程束大小为 $W = 32$ ，线程块中线性化索引为 t 的线程属于

$$\text{warpId} = \left\lfloor \frac{t}{W} \right\rfloor.$$

因此，索引 0 ~ 31 属于线程束 0，索引 32 ~ 63 属于线程束 1，依此类推。一个包含 T_b 个线程的线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。

例题：三个 256 线程的线程块

每个线程块包含

$$256/32 = 8$$

个线程束。若三个这样的线程块同时驻留在同一个流式多处理器上，则共有

$$8 \times 3 = 24$$

个驻留线程束。这里的 24 表示可供调度的线程束状态数量，不表示 24 个线程束在同一时钟周期同时发射指令。

4.2 处理块与线程束调度器

流式多处理器内部的执行核心还会组织成若干**处理块 (processing block, PB)**。在简化的流式多处理器模型中，每个处理块具有自己的控制与线程束调度资源；线程束被调度到这些处理块上执行。

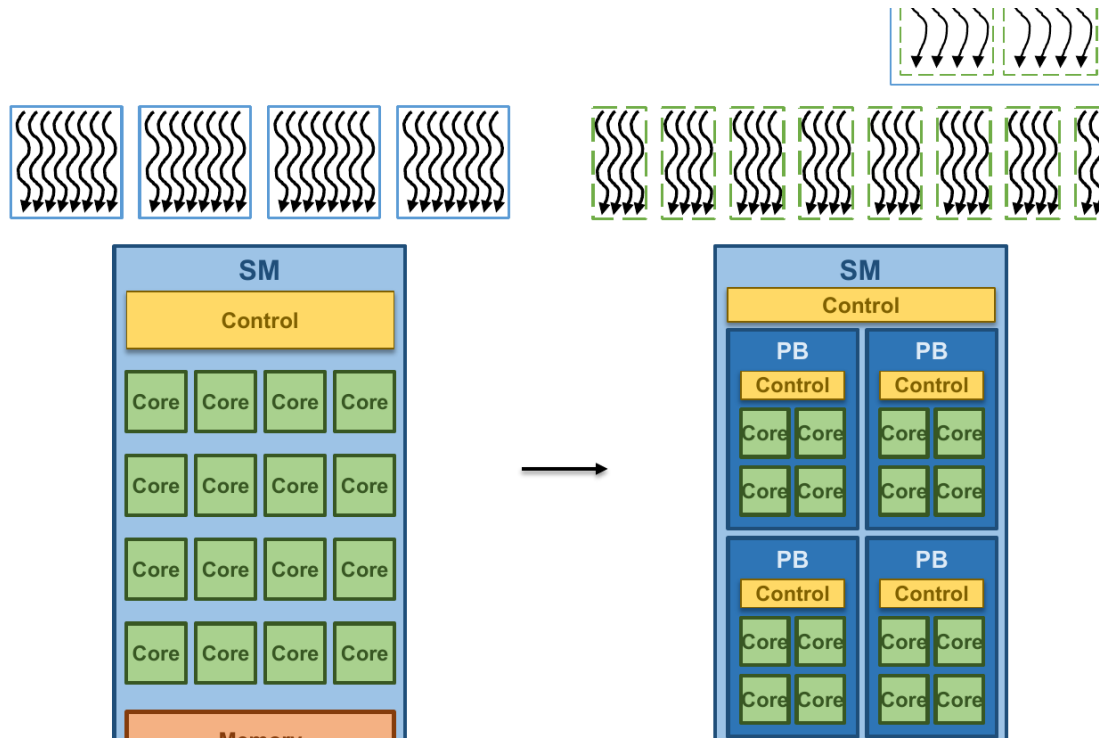


图3 线程块先拆分成线程束；流式多处理器内部再由多个处理块及其控制资源调度这些线程束

以 Hopper H100 为例，128 个 FP32 核心组织成 4 个处理块，每个处理块包含 32 个 FP32 核心并共享相应的指令分派资源。该图是吞吐结构的简化表示，因为一个流式多处理器内还存在整数单元、加载/存储单元、FP64 单元与张量核心等其他执行资源。

4.3 从单程序多数据到单指令多线程

软件层次的**单程序多数据** (*Single Program, Multiple Data, SPMD*) 描述的是：大量线程运行同一个核函数程序，每个线程依据自己的索引和数据完成不同实例的工作。

硬件调度层次则使用**单指令多线程** (*Single Instruction, Multiple Threads, SIMT*)：当某个线程束被选中时，调度器为该线程束发射一条指令，各个活动线程在各自的数据和线程状态上执行这条指令。该组织与**单指令多数据** (*Single Instruction, Multiple Data, SIMD*) 具有相同的控制复用思想，即让多个执行通道共享指令获取和分派成本。

关键结论

SPMD 是程序员看到的核函数编程模型；SIMT 是线程束层次的硬件执行模型。二者的连接点是线程束：来自同一核函数的线程被成组调度，同一时刻的指令发射以线程束为单位。

这种结构的主要硬件收益是摊薄控制成本。若 32 个执行通道都要执行同一条指令，就不需要为每个通道复制完全独立的指令获取与分派逻辑。代价则是：如果线程束内部的线程需要走不同控制路径，执行通道不能始终保持同时有效，这就产生控制发散。

4.4 独立线程调度不改变性能分析的基本单位

从 Volta 开始，NVIDIA 引入**独立线程调度** (*independent thread scheduling*)。线程束中的线程可以维护更独立的执行状态，在发散后沿不同路径前进并在之后重新会合，不再要求所有线程在所有情况下表现为严格锁步。

性能分析仍把线程束视为关键单位，原因有两点：第一，执行资源和指令发射仍围绕线程束组织；第二，线程束内部的控制不一致依然会降低有效通道利用率。独立线程调度还意味着某些过去依赖隐含锁步行为的线程间协作必须使用更明确的同步机制。

4.5 驻留线程束、就绪线程束与实际发射线程束

对一个流式多处理器中的线程束，需要区分三个状态概念：

- **驻留线程束**：其线程状态、寄存器等资源已经分配在流式多处理器中；
- **就绪或可选线程束**：下一条指令所需操作数与执行条件已经满足，可以被调度器选择；
- **实际发射线程束**：当前获得发射机会、正在向执行资源送出指令的线程束。

占用率统计的是驻留规模，而不是“这一周期有多少线程束正在发射”。理解这一差别，是后续延迟隐藏和占用率分析的前提。

5 控制发散与有效执行通道

5.1 控制发散是线程束内部的问题

当同一线程束中的线程对分支条件得到不同结果，或执行不同数量的循环迭代时，就出现**控制发散** (*control divergence*)。线程束需要依次处理不同控制路径；处理某一路径时，只启用需要执行该路径的通道，其他通道处于非活动状态。

如果两个不同线程束分别选择不同路径，但每个线程束内部的 32 个线程都一致，则不存在该意义下的线程束内发散。因而判断发散时，关键问题是：**同一线程束的活动通道是否对当前控制条件意见一致。**

5.2 分支发散示例

考虑以下条件：

代码 2 线程束内的分支发散

```
1 if (threadIdx.x < 24) {  
2     A;  
3 } else {  
4     B;  
5 }  
6 C;
```

对第一个线程束，线程索引为 0 ~ 31。条件成立的线程有 24 个，条件不成立的线程有 8 个。因此执行路径 A 时有 24 个活动通道、8 个非活动通道；执行路径 B 时则相反。两条路径都完成后，线程束重新会合并共同执行 C。

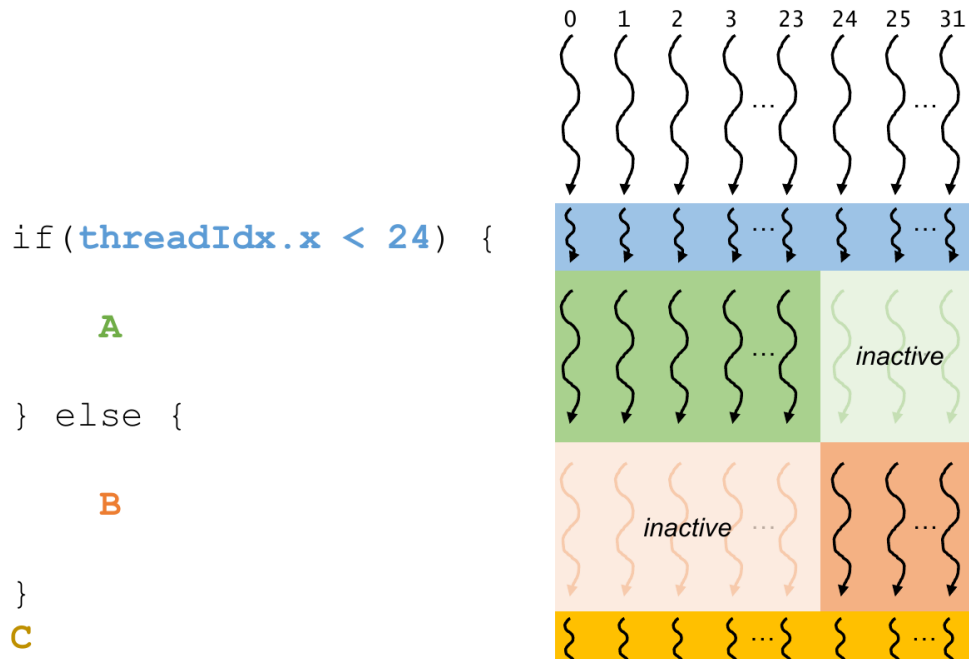


图 4 同一线程束中的线程在条件分支上分成两组时，两条路径对该线程束串行处理，未选择当前路径的通道保持非活动

5.3 活动通道比例与发散区间效率

活动通道比例 (active-lane fraction) 是一次 SIMT/SIMD 执行中启用的线程或执行通道所占比例。对于线程束大小 W ，某条路径上有 n 个活动线程时，活动通道比例为

$$f_{\text{active}} = \frac{n}{W}.$$

在上例中，路径 A 的活动通道比例是 $24/32 = 75\%$ ，路径 B 的活动通道比例是 $8/32 = 25\%$ 。若两条路径拥有相同数量的动态指令，则仅考虑分支区间时，两个串行阶段一共提供 $2W = 64$ 个通道-指令槽，其中实际有用的通道-指令数为 $24 + 8 = 32$ ，因此平均有效比例为

$$\eta = \frac{24 + 8}{2 \times 32} = 50\%.$$

若不同路径长度不同，可把这一思路写成加权形式。设路径 p 有 n_p 个活动线程，并执

行 I_p 条动态指令，则发散区间的通道利用率可写成

$$\eta = \frac{\sum_p n_p I_p}{W \sum_p I_p}.$$

该表达式只描述“每条路径串行执行、非参与通道关闭”造成的通道利用率，没有包含存储延迟、流水线吞吐或其他瓶颈。

5.4 循环也会产生控制发散

若每个线程的循环次数来自线程私有数据，例如

代码 3 每线程循环次数不同造成的发散

```
1 int N = a[threadIdx.x];
2 for (int i = 0; i < N; ++i) {
3     A;
4 }
```

则线程束中的线程可能在不同迭代结束。只要还有某个活动线程尚未达到自己的 N ，线程束就必须继续执行循环体；已经完成的线程通道则保持非活动。随着越来越多线程退出，活动通道比例会逐步下降。

因此，发散不仅来自显式的 **if/else**。任何依赖每线程数据的控制结构，包括循环退出条件，都可能让同一线程束内部的动态执行路径不同。

5.5 通过线程束粒度组织条件减少发散

一种减少分支发散的方法是让条件在一个线程束内部保持恒定。例如令条件依赖

$$\left\lfloor \frac{\text{threadIdx.x}}{\text{warpSize}} \right\rfloor,$$

则同一线程束内的线程具有相同商值。不同线程束仍然可以选择不同分支，但每个线程束内部不会因为该条件产生发散。

代码 4 线程束粒度的分支条件

```
1 if (threadIdx.x / warpSize > 2) {
2     // 路径 A
3 } else {
4     // 路径 B
5 }
```

这里仍有两个程序路径；优化点在于路径选择发生在线程束粒度，而不是在线程粒度。

5.6 边界检查中的少量发散通常不可避免

处理向量或图像时，问题规模经常不是线程块大小的整数倍。常见写法会向上取整线程块数量，然后在核函数中执行边界检查：

代码 5 向量尾部的边界检查

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 if (i < N) {
3     // 有效工作
4 }
```

最后一个线程块中可能只有部分线程有效，因此最后若干线程束会出现发散。这类发散通常只影响计算尾部的一小部分线程束，代价有限。实际目标不是机械消除所有分支，而是避免让大量线程束长期处于严重的通道失活状态。

关键结论

减少控制发散的核心原则是让控制条件尽量在线程束内部保持一致。判断性能影响时，应关注发散路径的长度以及每条路径上的活动通道比例，而不是只统计源代码里有多少个分支语句。

6 线程束调度与延迟隐藏

6.1 线程束调度器选择可执行的线程束

流式多处理器同时维护多个驻留线程束。在线程束已有驻留状态之间切换发射机会，可近似视为**零开销线程束调度 (zero-overhead warp scheduling)**：当前线程束无法继续时，调度器可以迅速选择另一个已经就绪的驻留线程束，无需执行传统 CPU 线程上下文切换所需的完整保存与恢复。

线程束只有在下一条指令具备执行条件时才能成为候选对象，其中一个关键条件是所需操作数已经可用，即操作数已 “ready for consumption”。

机制

每个驻留线程已经在流式多处理器的寄存器文件中占有相应状态；线程束还具有程序计数器等控制状态。在线程束之间切换发射机会时，不需要把整个线程上下文先写回内存再恢复。调度器主要是在已有驻留状态之间选择下一批要推进的线程束。

CUDA 不向程序员暴露具体的线程束优先级策略。程序可以依赖的是：满足执行条件的驻留线程束会被硬件调度以持续推进，而不能编写依赖某个固定线程束优先顺序的程序。

6.2 长延迟操作如何造成线程束停顿

算术指令所需操作数通常要先进入寄存器。若线程束执行高延迟的全局内存加载，而后续指令立即依赖加载结果，那么在数据返回之前，该线程束无法继续执行依赖指令，因而处于等待状态。

类似地，某些需要多个周期的执行资源也可能造成依赖停顿。关键并不在于停顿来源是内存还是计算，而在于：当前线程束的下一条指令暂时无法发射。

6.3 用其他线程束覆盖等待时间

延迟隐藏(*latency hiding*)的核心是把一个线程束的等待区间与其他线程束的有用执行重叠。设线程束 W_0 发出高延迟操作后必须等待；只要同一流式多处理器中存在另一个已经就绪的线程束 W_1 ，调度器就可以让 W_1 获得执行机会。若 W_1 之后也停顿，则继续寻找 W_2 、 W_3 等可执行线程束。

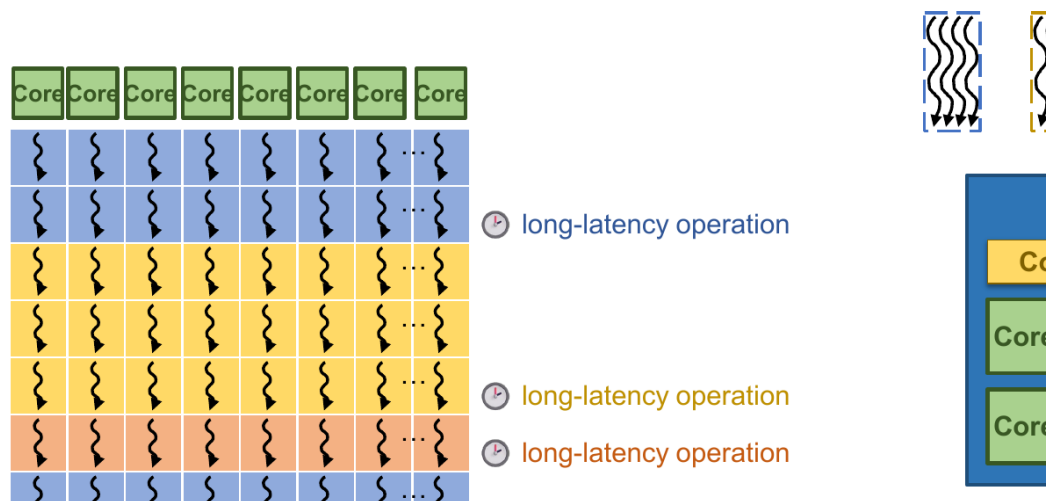


图 5 某个线程束遇到高延迟操作后，处理块可以选择其他就绪线程束继续执行，从而把等待时间与有用工作重叠

延迟隐藏不缩短一次全局内存访问本身需要的时间；它通过在这段时间内执行其他独立工作来减少执行资源的空闲时间。因而判断它是否有效时，应问“等待期间有没有别的线程束可以推进”，而不是只问“单次操作的延迟是多少”。

6.4 为什么驻留线程数远大于物理核心数

一个流式多处理器通常支持的最大驻留线程数明显大于其物理执行核心数，这使硬件能够保持较大的候选工作池。

假设流式多处理器中只有很少线程束。如果这些线程束同时发出长延迟加载，调度器就可能找不到任何就绪线程束，执行资源只能空等。若驻留线程束数量更多，则在部分线程束等待时，其他线程束已经准备好操作数的概率通常更高。

因此可以把关系概括为

更多驻留线程束 $\implies$ 更大的就绪候选集合 $\implies$ 更强的延迟隐藏机会。

这条关系解释了为什么占用率常常重要：占用率提高意味着流式多处理器上同时驻留的线程束更多，从而更可能在某个线程束停顿时找到替代工作。

6.5 延迟隐藏失效的典型情形

延迟隐藏只有在存在其他可执行线程束时才有作用。以下情况会使“增加线程束”无法继续解决问题：

- **内存带宽已经饱和。**更多线程束只会增加等待请求，不能提高已经受带宽上限约束的吞吐；
- **所有线程束都受到同类未解决依赖阻塞。**如果没有任何线程束的下一条指令已经就绪，调度器仍然无事可做；
- **缺少足够的指令级并行。**指令级并行 (*instruction-level parallelism*) 不足时，各线程束可能都很快进入依赖停顿，无法形成可交错执行的独立工作。

关键结论

延迟隐藏的必要条件不是“高占用率”这个数字本身，而是驻留工作中持续存在可发射的独立线程束。高占用率通常提高这一机会，但不能越过带宽、依赖或执行资源的硬瓶颈。

7 占用率、资源约束与线程块大小

7.1 占用率的定义

占用率 (occupancy) 描述一个流式多处理器上实际驻留的线程束数量，相对于该流式多处理器允许的最大驻留线程束数量的比例。记实际驻留线程束数为 W_{res} ，最大驻留线程束数为 W_{max} ，则

$$\mathcal{O} = \frac{W_{\text{res}}}{W_{\text{max}}}.$$

由于同一架构上的线程束大小固定，若只考虑线程数约束，也可以等价地用驻留线程数与最大驻留线程数之比理解。但正式定义以驻留线程束为中心更直接，因为调度器的候选单位就是线程束。

需要特别注意

占用率中的“驻留/活动”不等于“当前周期正在执行”。一个线程束可以已经驻留，却正在等待内存；也可以已经就绪，却暂时没有得到发射机会。占用率衡量的是可驻留并保持状态的工作池大小。

7.2 流式多处理器资源示例

不同 GPU 架构具有不同的资源参数。最大驻留线程数通常显著高于每个流式多处理器的核心数，同时还受到线程块数、寄存器和共享内存等约束。

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

图 6 多代 GPU 的流式多处理器资源对比；红框突出每个流式多处理器的核心数与最大驻留线程数

GPU	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
流式多处理器数量	15	24	56	80	108	132
每个流式多处理器核心数	192	128	64	64	64	128
最大驻留线程数/流式多处理器	2048	2048	2048	2048	2048	2048
最大驻留线程块数/流式多处理器	16	32	32	32	32	32
最大线程数/线程块	1024	1024	1024	1024	1024	1024
寄存器/流式多处理器	64K	64K	64K	64K	64K	64K
共享内存/流式多处理器	48KB	96KB	64KB	96KB	164KB	228KB

表 1 多代 GPU 的流式多处理器资源示例

7.3 把多重驻留限制写成统一形式

设一个线程块有 T_b 个线程，线程束大小为 W ，则每个线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。设设备限制分别为：最大驻留线程块数 $B_{\max}$ 、最大驻留线程数 $T_{\max}$ 、最大驻留线程束数 $W_{\max}$ 。若每个线程块还需要 R_b 个寄存器资源单位与 S_b 字节共享内存，而流式多处理器总量分别为 R_{SM} 和 S_{SM} ，则多重限制可概念化为

$$B_{\text{res}} \leq \min \left(B_{\max}, \left\lceil \frac{T_{\max}}{T_b} \right\rceil, \left\lceil \frac{W_{\max}}{W_b} \right\rceil, \left\lceil \frac{R_{\text{SM}}}{R_b} \right\rceil, \left\lceil \frac{S_{\text{SM}}}{S_b} \right\rceil \right).$$

该表达式用于统一理解各类驻留约束；实际硬件还可能按特定粒度分配寄存器或共享内存，因此精确驻留数应由设备属性与占用率工具计算。确定 B_{res} 后，驻留线程束数为

$$W_{\text{res}} = B_{\text{res}} W_b,$$

再代入占用率定义即可。

7.4 H100 风格限制下的三个线程块大小

以下例题采用一组简化限制：每线程块最多 1024 个线程，每流式多处理器最多 2048 个驻留线程、最多 32 个驻留线程块，并假设寄存器和共享内存不形成额外限制。

7.4.1 每块 256 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{256} \right\rfloor = 8$$

个线程块，且 $8 < 32$ 。所以可驻留 $8 \times 256 = 2048$ 个线程，即 64 个线程束；在这一简化模型中占用率为

$$\mathcal{O} = \frac{64}{64} = 100\%.$$

7.4.2 每块 32 个线程

若只看线程数，需要

$$2048/32 = 64$$

个线程块才能填满线程容量，但设备最多只允许 32 个驻留线程块。因此实际只能驻留

$$32 \times 32 = 1024$$

个线程，也就是 32 个线程束，对应简化占用率

$$\mathcal{O} = \frac{32}{64} = 50\%.$$

这里的瓶颈是最大驻留线程块数。

7.4.3 每块 768 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{768} \right\rfloor = 2$$

个线程块，驻留线程数为

$$2 \times 768 = 1536.$$

剩余 $2048 - 1536 = 512$ 个线程容量无法再容纳完整的第三个线程块，因此简化占用率为

$$\mathcal{O} = \frac{1536}{2048} = 75\%.$$

关键结论

线程块驻留必须以完整线程块为单位，所以资源利用会出现“装箱”效应。即使总资源还有余量，只要余量不足以容纳另一个完整线程块，这部分资源就不能被该核函数的额外线程块使用。

7.5 二维线程块大小：彩色转灰度核函数

另一个例题假设某 GPU 的每个流式多处理器最多驻留 1536 个线程和 8 个线程块，比较 8×8 、 16×16 与 32×32 三种二维线程块。

线程块形状	每块线程数	可驻留块数	驻留线程数	简化占用率
8×8	64	8	512	33.3%
16×16	256	6	1536	100%
32×32	1024	1	1024	66.7%

推导过程如下：

- 8×8 ：线程数允许 $1536/64 = 24$ 个线程块，但块数上限为 8，所以只能驻留 512 个线程；
- 16×16 ： $1536/256 = 6$ 个线程块，既不超过 8 块限制，又正好使用全部 1536 个线程容量；
- 32×32 ：每块 1024 线程，第二块会使线程数达到 2048，因此只能驻留一块。

在该题给定的简化限制下， 16×16 获得最高理论占用率。

7.6 占用率高并不保证性能最高

较高占用率通常有利于增加延迟隐藏机会，但占用率只在能够帮助隐藏延迟的范围内有用，并不单独保证性能。

原因与上一节一致：若内存带宽已经饱和、所有线程束都受共同依赖阻塞，或其他执行资源成为瓶颈，即使继续增加驻留线程束，也未必增加有效指令吞吐。因此，占用率最适合作为资源驻留与延迟隐藏能力的诊断量，而不是完整的性能模型。

8 运行时资源查询与例题

8.1 运行时查询设备属性

不同 GPU 架构的资源上限不同，因此程序不应把所有驻留限制都永久写死。CUDA 运行时提供 `cudaGetDeviceProperties` 查询设备属性：

代码 6 `cudaGetDeviceProperties` 函数原型

```
1 cudaError_t cudaGetDeviceProperties(cudaDeviceProp* prop, int device);
```

基本用法如下：

代码 7 查询设备线程资源上限

```
1 int devID = 0;  
2 cudaDeviceProp devProp;  
3 cudaGetDeviceProperties(&devProp, devID);  
4  
5 int maxThreadsPerBlock = devProp.maxThreadsPerBlock;  
6 int maxThreadsPerMultiProcessor =  
7     devProp.maxThreadsPerMultiProcessor;
```

设备属性结构还可以提供线程束大小以及其他架构限制。程序可以据此选择更合适的启动配置，而不是假设未来设备与当前设备拥有完全相同的线程数、线程块数或线程束限制。

8.2 占用率计算工具

NVIDIA Nsight Compute 提供**占用率计算器 (Occupancy Calculator)**, 能够根据目标 GPU、每线程块线程数、每线程寄存器使用量、共享内存使用量等信息估算多处理器占用率。该工具替代了较早的 CUDA 占用率计算表格。

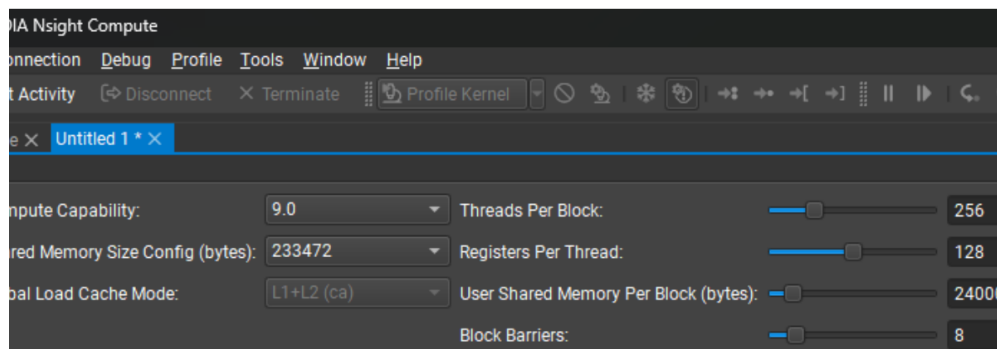


图 7 占用率计算器把线程块大小、寄存器和共享内存等静态资源需求映射为驻留线程束与线程块限制

占用率工具的价值在于快速识别明显的资源浪费或驻留瓶颈。它不能替代性能测量，因为高占用率只说明潜在的延迟隐藏能力较强，并不能说明实际瓶颈一定来自驻留线程束不足。

8.3 例题一：1536 线程与 4 线程块上限

某 CUDA 设备的每个流式多处理器最多驻留 1536 个线程，并且最多驻留 4 个线程块。比较每块 256、384、576 个线程，哪一种配置能在仅考虑这两个限制时填满线程容量？

完整推导

每块 256 线程：

$$\frac{1536}{256} = 6.$$

线程数允许 6 个线程块，但块数上限只有 4，因此实际最多驻留

$$4 \times 256 = 1024$$

个线程，不能填满 1536 线程容量。

每块 384 线程：

$$\frac{1536}{384} = 4.$$

恰好需要 4 个线程块，既满足线程数限制，也满足线程块数限制，驻留线程数为

$$4 \times 384 = 1536.$$

因此该配置在题目给定的简化限制下达到满线程容量。

每块 576 线程：

$$\left\lfloor \frac{1536}{576} \right\rfloor = 2,$$

只能驻留

$$2 \times 576 = 1152$$

个线程；第三个线程块会使线程总数超过 1536，所以不能填满容量。

因此答案是 **384 线程/块**。

8.4 例题二：只知道流式多处理器和核心数，能否选出最佳配置

题目给定一个一维数组，每个元素由一个线程处理。目标 GPU 有 8 个流式多处理器，每个流式多处理器有 16 个流式处理器，问“最佳执行配置”是什么。

仅凭这些信息无法确定。流式多处理器数量和核心数只描述部分物理执行资源；至少还缺少：

- 每流式多处理器最大驻留线程数和最大驻留线程块数；
- 每线程块最大线程数；
- 寄存器与共享内存容量；
- 该核函数每线程寄存器使用量和每线程块共享内存使用量；
- 实际性能测量，用于判断瓶颈是否来自占用率、带宽或其他资源。

关键结论

从“核心数”直接推导线程块大小是不充分的。CUDA 的启动配置同时受到驻留限制和核函数资源足迹约束，而最高理论占用率也不等价于最高实测性能。

8.5 例题三：统一分支条件是否造成控制发散

考虑核函数：

代码 8 统一分支条件的核函数

```
1 __global__ void do_work(int q, int *A) {  
2     int result = 0;  
3     if (q < 5)  
4         result = threadIdx.x;  
5     A[threadIdx.x] = result;  
6 }
```

问题是：条件 $q < 5$ 是否造成控制发散？

答案是**不会**。变量 q 是统一传入的标量参数，同一线程束中的线程对该条件得到相同结果。因此整个线程束要么都执行赋值语句，要么都跳过赋值语句。虽然赋值结果使用了每线程不同的 threadIdx.x ，但这只意味着各线程处理不同数据，并不意味着控制路径不同。

更一般地说，控制发散要求同一线程束的活动线程对分支谓词得到不同结果。分支条件不必显式包含线程索引才可能发散；只要它依赖的每线程数据不同，就可能造成发散。反之，即使分支语句存在，只要谓词在线程束内一致，就没有线程束内控制发散。

9 架构、调度与性能机制的统一关系

9.1 从网格到执行通道的完整映射

从软件层次到硬件执行资源的完整映射链为

网格 → 线程块 → 流式多处理器驻留
→ 线程束 → 处理块调度 → 执行通道

线程块决定资源分配和协作范围；线程束决定调度与控制发散的粒度；处理块和执行资源决定线程束指令如何获得实际执行机会。把这三个粒度区分开，就能避免把“线程块同时驻留”“线程束被调度”和“线程在核心上执行”混为同一件事。

9.2 四个性能概念之间的因果链

1. **资源限制决定驻留规模。**线程数、线程块数、寄存器和共享内存等限制共同决定一个流式多处理器能容纳多少线程块与线程束。
2. **驻留规模决定占用率。**驻留线程束数与架构最大线程束数之比就是占用率。
3. **占用率影响延迟隐藏机会。**驻留线程束越多，某个线程束停顿时找到另一个就绪线程束的机会通常越高。
4. **控制发散降低单个线程束的有效通道利用。**即使有足够多驻留线程束，如果每个线程束内部大量通道因为分支而失活，执行资源的有效利用仍会下降。

因此，线程块大小不是一个只影响“线程有多少”的参数。它同时改变每块线程束数、流式多处理器可驻留线程块数、资源装箱效率和可用于延迟隐藏的线程束池；如果算法中的控制结构又与线程索引相关，还要进一步考虑线程束内部的发散模式。

9.3 概念边界总结

概念	主要粒度	回答的问题
线程块驻留	线程块/流式多处理器	一个线程块能否进入某个流式多处理器，以及同时能驻留多少块
线程束调度	线程束/处理块	哪个已经驻留且就绪的线程束现在获得指令发射机会
控制发散	单个线程束内部	同一线程束的活动线程是否沿不同控制路径执行
延迟隐藏	多个驻留线程束	当前线程束等待时，是否有其他就绪线程束可以使用执行资源
占用率	流式多处理器	驻留线程束池相对架构最大容量有多大
线程块内同步	单个线程块	同一线程块中的线程如何在共享资源上建立协作和屏障
跨线程块协作	多线程块/更高作用域	如何在不依赖普通线程块调度顺序的前提下建立受约束的协作